



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

Facultad de Estudios Profesionales Zona Media

PRÁCTICA 11

Medición de temperatura y control de ventilador con monitoreo gráfico

Sensor LM35, ADC, PWM y visualización con Python

Alumno: Cesar Gael Mancilla Estrada
Clave: 382031
Carrera: Ingeniería en Mecatrónica
Materia: Microcontroladores
Profesor: Ing. Jesús Padrón
Fecha: Marzo de 2025

"La temperatura es una de las variables más medidas en la industria"

Índice

1. Objetivo	2
2. Requisitos y Material	2
2.1. Hardware necesario	2
2.2. Características del LM35	2
2.3. Conexiones del circuito	3
3. Desarrollo	3
3.1. Diagrama en Protoboard	3
3.2. CMakeLists.txt	3
3.3. Código C para la Pico W	4
3.4. Código Python para visualización	6
4. Funcionamiento del Sistema	10
5. Preguntas de Repaso	10
6. Conclusiones	11
6.1. Aprendizajes de la práctica	11
6.2. Posibles mejoras	11
7. Referencias	12

1. Objetivo

Aprender a utilizar el convertidor analógico-digital (ADC) del microcontrolador Raspberry Pi Pico para medir temperatura mediante un sensor analógico LM35, controlar un ventilador por PWM y visualizar los datos obtenidos mediante una gráfica generada en Python utilizando matplotlib.

2. Requisitos y Material

2.1. Hardware necesario

Tabla 1: Material requerido

Componente	Cantidad
Raspberry Pi Pico W	1
Sensor LM35	1
Ventilador de computadora (5V/12V)	1
Transistor 2N2222A	1
Resistencia 1kΩ	2
LED rojo	1
LED verde	1
Zumbador activo	1
Protoboard	1
Cables jumper	Varios

2.2. Características del LM35

El LM35 es un sensor de temperatura que entrega un voltaje proporcional a la temperatura:

$$\text{Temperatura (}^{\circ}\text{C)} = \text{Voltaje (mV)} \times 100$$

Relación: **10 mV por cada grado Celsius.**

2.3. Conexiones del circuito

Tabla 2: Conexiones del sistema

Componente	Pin LM35/Pico	GPIO
LM35 - VCC	3.3V	-
LM35 - GND	GND	-
LM35 - OUT	GP26	ADC0
Ventilador (Base transistor)	GP15	PWM
LED verde	GP12	Salida digital
Zumbador	GP13	Salida digital
LED rojo	GP14	Salida digital
LED interno	GP25	Indicador

3. Desarrollo

3.1. Diagrama en Protoboard

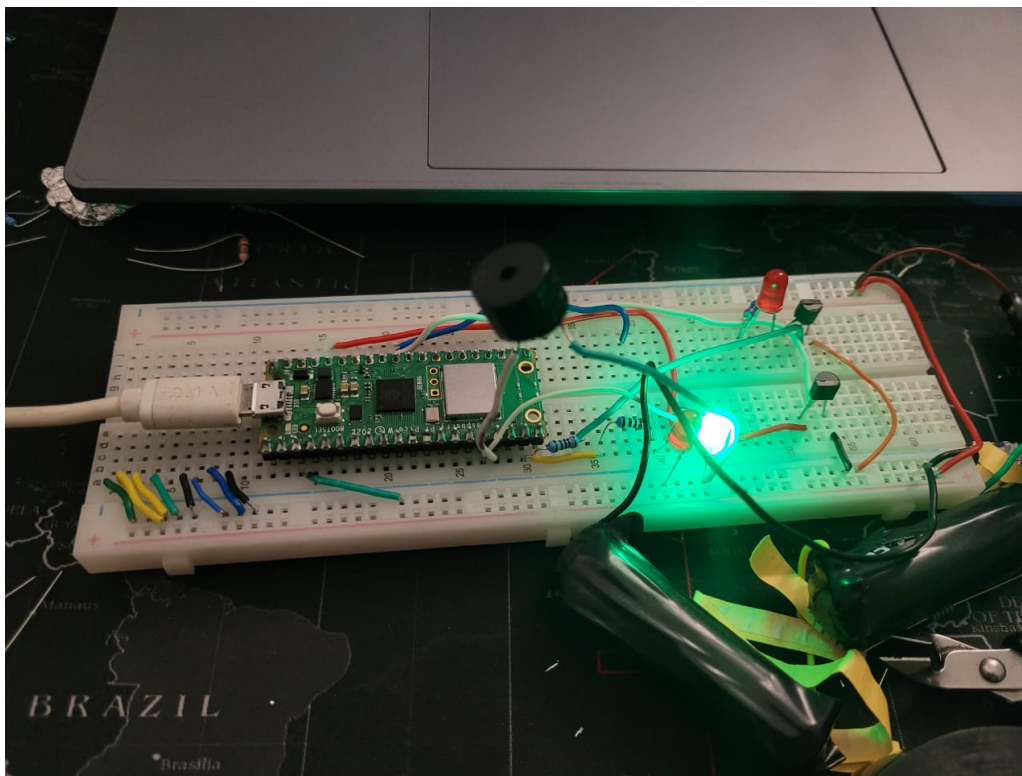


Figura 1: Conexión del sensor LM35, LEDs, zumbador y ventilador en protoboard

3.2. CMakeLists.txt

Listing 1: Archivo CMakeLists.txt

```
1 cmake_minimum_required(VERSION 3.13)
```

```
2
3 include(pico_sdk_import.cmake)
4 set(PICO_BOARD pico_w)
5
6 project(Practica_11)
7
8 pico_sdk_init()
9
10 add_executable(temperatura
11     temperatura.c
12 )
13
14 target_link_libraries(temperatura
15     pico_stdlib
16     hardware_adc
17     hardware_gpio
18     hardware_pwm
19 )
20
21 pico_enable_stdio_usb(temperatura 1)
22 pico_enable_stdio_uart(temperatura 0)
23
24 pico_add_extra_outputs(temperatura)
```

3.3. Código C para la Pico W

Listing 2: temperatura.c - Lectura del LM35 y control de ventilador

```
1 /*
2  * Practica 11: Medicion de temperatura con LM35
3  * Sensor: LM35 (10mV/ C )
4  * Ventilador: Control por PWM con transistor 2N2222
5  */
6
7 #include <stdio.h>
8 #include "pico/stdlib.h"
9 #include "hardware/adc.h"
10 #include "hardware/gpio.h"
11 #include "hardware/pwm.h"
12
13 #define LM35_PIN 26          // GP26 = ADC0
14 #define FAN_PIN 15           // GP15 para control PWM
15 #define LED_VERDE_PIN 12     // GP12 - Temperatura normal
16 #define ZUMBADOR_PIN 13     // GP13 - Alarma sonora
17 #define LED_ROJO_PIN 14     // GP14 - Alarma visual
18
19 #define ADC_MAX_VALUE 4095
20 #define ADC_VREF 3.3f
21 #define TEMP_UMBRAL 45.0f    // Umbral para activar ventilador
22
23 void init_adc(void);
```

```
24 void init_pwm(void);
25 void init_indicadores(void);
26 float leer_temperatura(void);
27 void controlar_ventilador(float temp);
28 void actualizar_indicadores(float temp);
29
30 int main() {
31     stdio_init_all();
32     init_adc();
33     init_pwm();
34     init_indicadores();
35
36     printf("=====\n");
37     printf("Practica 11: Medicion de Temperatura\n");
38     printf("Sensor: LM35\n");
39     printf("Umbral: %.1f C\n", TEMP_UMBRAL);
40     printf("=====\n\n");
41
42     while (true) {
43         float temperatura = leer_temperatura();
44
45         controlar_ventilador(temperatura);
46         actualizar_indicadores(temperatura);
47
48         printf("T: %.2f,\n", temperatura);
49
50         sleep_ms(500);
51     }
52     return 0;
53 }
54
55 void init_adc(void) {
56     adc_init();
57     adc_gpio_init(LM35_PIN);
58     adc_select_input(0);
59 }
60
61 void init_pwm(void) {
62     gpio_set_function(FAN_PIN, GPIO_FUNC_PWM);
63     uint slice_num = pwm_gpio_to_slice_num(FAN_PIN);
64     pwm_set_wrap(slice_num, 255);
65     pwm_set_enabled(slice_num, true);
66     pwm_set_gpio_level(FAN_PIN, 0);
67 }
68
69 void init_indicadores(void) {
70     gpio_init(LED_VERDE_PIN); gpio_set_dir(LED_VERDE_PIN, GPIO_OUT)
71     ;
72     gpio_init(LED_ROJO_PIN);   gpio_set_dir(LED_ROJO_PIN, GPIO_OUT);
73     gpio_init(ZUMBADOR_PIN);   gpio_set_dir(ZUMBADOR_PIN, GPIO_OUT);
```

```

74     gpio_put(LED_VERDE_PIN, 1); // Verde encendido (normal)
75     gpio_put(LED_ROJO_PIN, 0); // Rojo apagado
76     gpio_put(ZUMBADOR_PIN, 0); // Zumbador apagado
77 }
78
79 float leer_temperatura(void) {
80     uint16_t raw = adc_read();
81     float voltaje = (raw * ADC_VREF) / ADC_MAX_VALUE;
82     return voltaje * 100.0f;
83 }
84
85 void controlar_ventilador(float temp) {
86     if (temp >= TEMP_UMBRAL) {
87         pwm_set_gpio_level(FAN_PIN, 255);
88         printf("VENTILADOR ON - Temp: %.2f C\n", temp);
89     } else {
90         pwm_set_gpio_level(FAN_PIN, 0);
91     }
92 }
93
94 void actualizar_indicadores(float temp) {
95     if (temp >= TEMP_UMBRAL) {
96         gpio_put(LED_VERDE_PIN, 0); // Verde apagado
97         gpio_put(LED_ROJO_PIN, 1); // Rojo encendido
98         gpio_put(ZUMBADOR_PIN, 1); // Zumbador suena
99     } else {
100         gpio_put(LED_VERDE_PIN, 1); // Verde encendido
101         gpio_put(LED_ROJO_PIN, 0); // Rojo apagado
102         gpio_put(ZUMBADOR_PIN, 0); // Zumbador apagado
103     }
104 }

```

3.4. Código Python para visualización

Listing 3: grafica_temperatura.py - Visualización en tiempo real

```

1  """
2  Monitoreo de temperatura con interfaz grafica
3  Sensor LM35 + Raspberry Pi Pico W
4  """
5
6  import tkinter as tk
7  from tkinter import ttk
8  import serial
9  import matplotlib.pyplot as plt
10 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
11 import matplotlib.animation as animation
12 import threading
13 import time
14 import re
15

```

```
16 class MonitorTemperatura:
17     def __init__(self, puerto='COM3', baudrate=115200):
18         self.puerto = puerto
19         self.baudrate = baudrate
20         self.serial_activo = False
21         self.ser = None
22
23         # Datos
24         self.tiempos = []
25         self.temperaturas = []
26         self.max_puntos = 200
27         self.tiempo_inicio = None
28
29         # Patron para extraer temperatura
30         self.patron = re.compile(r'T:([0-9.]+)')
31
32         # Ventana principal
33         self.root = tk.Tk()
34         self.root.title("Monitor de Temperatura - LM35")
35         self.root.geometry("1000x600")
36
37         self.crear_interfaz()
38         self.configurar_grafica()
39
40     def crear_interfaz(self):
41         # Frame de controles
42         frame_controles = ttk.Frame(self.root, padding=10)
43         frame_controles.pack(fill=tk.X)
44
45         ttk.Label(frame_controles, text="Puerto:").pack(side=tk.
46             LEFT, padx=5)
47         self.entry_puerto = ttk.Entry(frame_controles, width=10)
48         self.entry_puerto.pack(side=tk.LEFT, padx=5)
49         self.entry_puerto.insert(0, self.puerto)
50
51         self.btn_conectar = ttk.Button(frame_controles, text="
52             Conectar",
53             command=self.conectar)
54         self.btn_conectar.pack(side=tk.LEFT, padx=5)
55
56         self.btn_desconectar = ttk.Button(frame_controles, text="
57             Desconectar",
58             command=self.desconectar,
59             state='disabled')
60         self.btn_desconectar.pack(side=tk.LEFT, padx=5)
61
62         self.label_estado = ttk.Label(frame_controles, text="
63             Desconectado",
64             foreground='red')
65         self.label_estado.pack(side=tk.LEFT, padx=20)
```



```
62     # Frame temperatura actual
63     frame_temp = ttk.Frame(self.root, padding=10)
64     frame_temp.pack(fill=tk.X)
65
66     self.label_temp = ttk.Label(frame_temp, text="-- C ",
67                                font=('Arial', 24, 'bold'))
68     self.label_temp.pack(side=tk.LEFT, padx=20)
69
70     self.label_stats = ttk.Label(frame_temp, text="Muestras:
71     0")
72     self.label_stats.pack(side=tk.LEFT, padx=20)
73
74     # Frame grafica
75     frame_grafica = ttk.Frame(self.root)
76     frame_grafica.pack(fill=tk.BOTH, expand=True, padx=10, pady
77                        =10)
78
79     self.fig, self.ax = plt.subplots(figsize=(10, 4))
80     self.ax.set_xlabel('Tiempo (s)')
81     self.ax.set_ylabel('Temperatura ( C )')
82     self.ax.set_title('Monitor de Temperatura Tiempo Real')
83     self.ax.grid(True, alpha=0.3)
84     self.ax.axhline(y=45, color='r', linestyle='--', label='
85     Umbral 45 C ')
86     self.ax.set_ylim(0, 100)
87     self.ax.set_xlim(0, 60)
88     self.ax.legend()
89
90     self.linea, = self.ax.plot([], [], 'b-', linewidth=2)
91     self.canvas = FigureCanvasTkAgg(self.fig, master=
92     frame_grafica)
93     self.canvas.draw()
94     self.canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
95
96     def configurar_grafica(self):
97         self.ani = animation.FuncAnimation(
98             self.fig, self.actualizar_grafica,
99             interval=200, cache_frame_data=False, blit=False
100         )
101
102     def conectar(self):
103         try:
104             self.puerto = self.entry_puerto.get()
105             self.ser = serial.Serial(self.puerto, self.baudrate,
106                                     timeout=0.1)
107             self.serial_activo = True
108             self.tiempo_inicio = time.time()
109
110             self.tiempos = []
111             self.temperaturas = []
```

```
108         self.label_estado.config(text="Conectado", foreground='
           green')
109         self.btn_conectar.config(state='disabled')
110         self.btn_desconectar.config(state='normal')
111
112         self.hilo = threading.Thread(target=self.leer_datos,
           daemon=True)
113         self.hilo.start()
114
115     except Exception as e:
116         self.label_estado.config(text=f"Error: {e}", foreground
           ='red')
117
118     def desconectar(self):
119         self.serial_activo = False
120         if self.ser:
121             self.ser.close()
122         self.label_estado.config(text="Desconectado", foreground='
           red')
123         self.btn_conectar.config(state='normal')
124         self.btn_desconectar.config(state='disabled')
125
126     def leer_datos(self):
127         while self.serial_activo:
128             try:
129                 if self.ser and self.ser.in_waiting:
130                     linea = self.ser.readline().decode('utf-8').
                       strip()
131                     match = self.patron.search(linea)
132                     if match:
133                         temp = float(match.group(1))
134                         t = time.time() - self.tiempo_inicio
135                         self.root.after(0, self.agregar_dato, t,
                           temp)
136             except:
137                 pass
138             time.sleep(0.01)
139
140     def agregar_dato(self, t, temp):
141         self.tiempos.append(t)
142         self.temperaturas.append(temp)
143
144         if len(self.tiempos) > self.max_puntos:
145             self.tiempos.pop(0)
146             self.temperaturas.pop(0)
147
148         self.label_temp.config(text=f"{temp:.2f} C ",
149                                foreground='red' if temp > 45 else '
150                                black')
```

```

151         self.label_stats.config(text=f"Muestras: {len(self.
            temperaturas)}")
152
153     def actualizar_grafica(self, frame):
154         if self.tiempos and self.temperaturas:
155             self.linea.set_data(self.tiempos, self.temperaturas)
156
157             if self.tiempos:
158                 t_max = max(self.tiempos)
159                 self.ax.set_xlim(max(0, t_max-60), t_max+1)
160
161             self.canvas.draw_idle()
162         return self.linea,
163
164     def iniciar(self):
165         self.root.mainloop()
166
167 if __name__ == "__main__":
168     import serial.tools.list_ports
169     puertos = [p.device for p in serial.tools.list_ports.comports()
170                ]
171     puerto = puertos[0] if puertos else 'COM3'
172     app = MonitorTemperatura(puerto=puerto)
173     app.iniciar()

```

4. Funcionamiento del Sistema

El sistema desarrollado realiza las siguientes tareas:

1. Lee el valor analógico del sensor LM35 usando el ADC de 12 bits
2. Convierte el valor digital a voltaje usando la fórmula: $V = \frac{raw \times 3,3}{4095}$
3. Calcula la temperatura: $T(^{\circ}C) = V \times 100$
4. Compara la temperatura con el umbral establecido ($45^{\circ}C$)
5. Si supera el umbral, activa el ventilador (PWM 100 %) y las alarmas
6. Envía los datos por USB para visualización en Python

5. Preguntas de Repaso

1. ¿Qué es un convertidor analógico-digital (ADC)?

Es un dispositivo que convierte una señal analógica (voltaje continuo) en un valor digital (número binario) que puede ser procesado por un microcontrolador.

2. ¿Qué resolución tiene el ADC del Raspberry Pi Pico?

El ADC del RP2040 tiene una resolución de **12 bits**, lo que significa que puede representar $2^{12} = 4096$ niveles diferentes (0 a 4095).

3. ¿Por qué es necesario convertir el valor digital a voltaje?

El ADC entrega un número entero proporcional al voltaje, pero no es directamente el voltaje. Se necesita convertir usando la fórmula para obtener el valor real en voltios.

4. ¿Cómo se obtiene la temperatura a partir del voltaje del LM35?

El LM35 entrega 10 mV por cada grado Celsius, por lo que: $T(C) = V_{out} \times 100$.

5. ¿Qué sucede si aumenta la resolución del ADC?

Aumenta la precisión de la medición. Por ejemplo, un ADC de 16 bits tendría 65536 niveles y una resolución de aproximadamente $50\mu V$ por nivel.

6. ¿Por qué es útil representar los datos mediante una gráfica?

Permite visualizar tendencias, identificar patrones, detectar anomalías y analizar el comportamiento del sistema en el tiempo.

7. ¿Qué aplicaciones industriales tiene la medición de temperatura con microcontroladores?

Control de procesos industriales, monitoreo de motores, sistemas HVAC, cámaras de refrigeración, hornos industriales, sistemas de seguridad contra incendios.

8. ¿Qué ventajas tiene registrar datos en el tiempo?

Permite análisis histórico, detección de tendencias, mantenimiento predictivo, optimización de procesos y documentación para normativas.

6. Conclusiones

6.1. Aprendizajes de la práctica

- Se comprendió el funcionamiento del ADC de 12 bits del RP2040
- Se aprendió a leer el sensor LM35 y convertir su salida a temperatura
- Se implementó control de ventilador por PWM usando transistor 2N2222
- Se desarrolló una interfaz gráfica en Python con matplotlib y tkinter
- Se implementó un sistema de alarmas visuales y sonoras

6.2. Posibles mejoras

- Implementar control PID para velocidad variable del ventilador
- Agregar registro de datos en archivo CSV
- Enviar datos a la nube (ThingSpeak, Firebase)
- Agregar notificaciones por correo o Telegram
- Usar display LCD para mostrar temperatura localmente

7. Referencias

1. Raspberry Pi Foundation. (2024). *Raspberry Pi Pico W Datasheet*.
2. Texas Instruments. (2024). *LM35 Datasheet - Precision Centigrade Temperature Sensors*.
3. Raspberry Pi Foundation. (2024). *Pico SDK Documentation - ADC and PWM*.